



Graphs and Traversals

CEN207 Data Structures — Week 5

Asst. Prof. Dr. Uğur CORUH · Fall 2026–2027

Today's plan (3 hours)

Hour	Topic
1	Why graphs, vocabulary Anim 1 · matrix vs. list Anim 2–3
2	BFS Anim 4 · DFS recursive Anim 5 · DFS iterative Anim 6
3	Connected components Anim 7 · shortest path Anim 8

Learning outcomes: LO.1 (explain fundamental data structures) · LO.2 (analyze complexity) · LO.7 (choose the right structure)

This week's concepts — where

Concept	Where
Why graphs, vocabulary	Sections 1–2
Adjacency matrix vs. adjacency list	Section 3
BFS, DFS (recursive and iterative)	Sections 4–6
Connected components, shortest path	Sections 7–8

How the code examples work

- Every idea has a complete **C** and **Java** program
- C: `gcc -std=c11 -Wall -Wextra -o /tmp/x file.c && /tmp/x`
- Java: `javac -d /tmp/j File.java && java -cp /tmp/j File`
- Sources: `code/week-05/c/` and `code/week-05/java/`
- Each program's expected output is in the week notes

Recap — stacks, queues, recursion (Week 3)

- A **stack** (LIFO): last pushed, first popped
- A **queue** (FIFO): first enqueued, first dequeued
- **Recursion** *is* a stack: a call pushes, a return pops
- Today: the very same stack and queue, holding graph vertices

Recap — trees are special graphs (Week 4)

- A tree: one root, no cycles, exactly one path to any node
- A graph: **no** root required, cycles allowed, many paths allowed
- Every tree **is** a graph; not every graph is a tree
- Traversals (pre/in/post/level) generalize to graph search today

Map of the week — at a glance

Foundations	Traversal & applications
Why graphs, vocabulary	BFS, DFS (recursive, iterative)
Matrix vs. list	Connected components
—	Shortest path (unweighted)

1. Why Graphs? From Bridges to Networks

A question to start

A city has four landmasses joined by seven bridges. Can you walk through the city crossing **every** bridge exactly once, and return home?

A short history

- **1736** — Leonhard Euler solves the Königsberg bridge puzzle
- Königsberg (today Kaliningrad): 4 landmasses, 7 bridges
- Euler's paper is the **first** graph-theory paper ever written
- He answered "no" — and explained exactly why, for any city

The bridges, restated

- Each **landmass** becomes a **vertex** (4 of them)
- Each **bridge** becomes an **edge** (7 of them)
- The walk becomes: trace every edge once, never lifting your pen
- This is an **Eulerian path** — named after Euler himself

Euler's insight: degree parity

- Every time you **pass through** a vertex, you use 2 edges
- Only your start and end vertex may use an **odd** number
- An Eulerian walk needs **0 or 2** odd-degree vertices, no more
- Königsberg has **4** odd-degree vertices — so no walk exists

Why it matters

- One puzzle, solved by counting, founded **graph theory**
- The same vertex/edge idea now models almost any relationship
- Modern uses: maps, social networks, the web, and much more
- The next three slides show three of them

Graphs are everywhere — road maps

- **Vertices:** intersections, cities, addresses
- **Edges:** roads, with a **weight** — distance or drive time
- GPS routing asks: shortest weighted path from A to B
- Week 9's Dijkstra's algorithm answers exactly that question

Graphs are everywhere — social networks

- **Vertices:** people or accounts
- **Edges:** "follows" (directed) or "is friends with" (undirected)
- "Friends of friends" — reachable in exactly 2 edges
- Section 4's BFS answers that question, one level at a time

Graphs are everywhere — the web

- **Vertices:** web pages; **edges:** hyperlinks (directed)
- **A>B** means page **A** links **to** page **B**
- A crawler visits every reachable page exactly once
- Section 6's DFS is a classic way to write a web crawler

Graph vs. tree, revisited

Tree (Week 4)	Graph (this week)
One root	No required root
No cycles allowed	Cycles allowed
Exactly one path between any two nodes	Zero, one, or many paths

Common mistakes

- Assuming a graph must be **connected** — it may not be
- Assuming an edge always goes **both ways** — only if undirected
- Forgetting a single vertex with no edges is still a valid graph

Mini-quiz

A city has exactly **two** odd-degree landmasses, the rest even. Can a walk crossing every bridge once exist? Where would it have to start?

Answer

Yes. With exactly two odd-degree vertices, an Eulerian **path** (not a full round trip) exists, starting at one odd vertex and ending at the other.

2. Graph Vocabulary

A question to start

A graph is just two sets: **V**, the vertices, and **E**, the edges. How much structure can really come from an idea that small?

A graph, formally

- A graph $G = (V, E)$: a set of vertices, a set of edges
- Each edge connects **two** vertices (possibly the same one)
- $|V|$ vertices, $|E|$ edges — common shorthand: V and E
- Everything below is built from just these two sets

Vocabulary (1/4)

Term	Meaning
Vertex (node)	A single point in the graph
Edge	A connection between two vertices
Undirected	Edge A-B usable in either direction
Directed	Edge A>B usable only A to B

Vocabulary (2/4)

Term	Meaning
Weighted	Every edge carries a number, its weight
Unweighted	Edges only say "connected", no number
Degree (undirected)	Edges touching a vertex

Vocabulary (3/4)

Term	Meaning
In-degree	Edges pointing into a vertex (directed)
Out-degree	Edges pointing out of a vertex (directed)
Path	An edge sequence, no repeated vertex
Cycle	A path that returns to its start

Vocabulary (4/4)

Term	Meaning
Connected	Every vertex reachable from every other
Self-loop	An edge from a vertex to itself
Multi-edge	More than one edge between the same pair

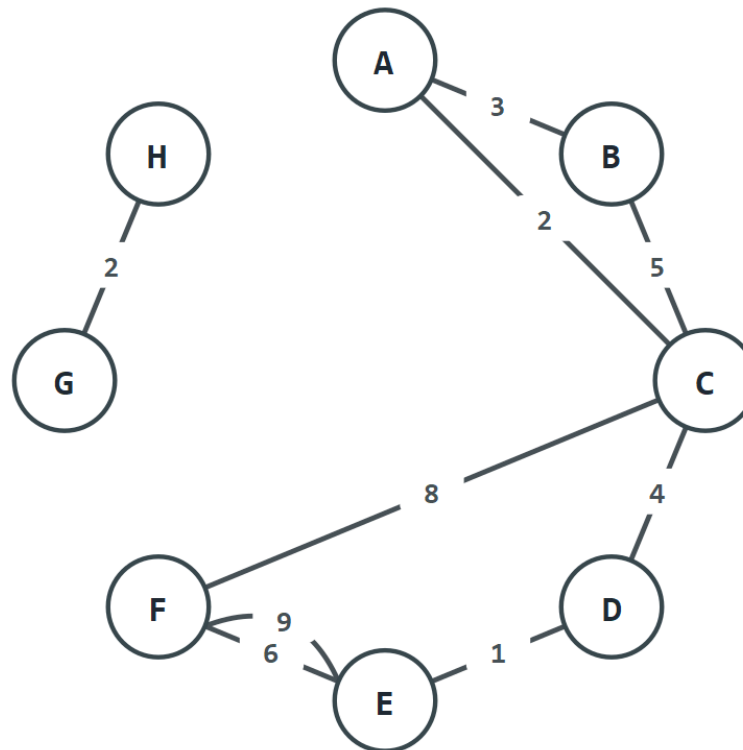
Sparse vs. dense

- **Sparse:** E close to V — few edges per vertex
- **Dense:** E close to V^2 — most pairs connected
- A road map: sparse (a city has few roads per intersection)
- A "who follows whom" graph on a small team: often dense

What can we ask a graph?

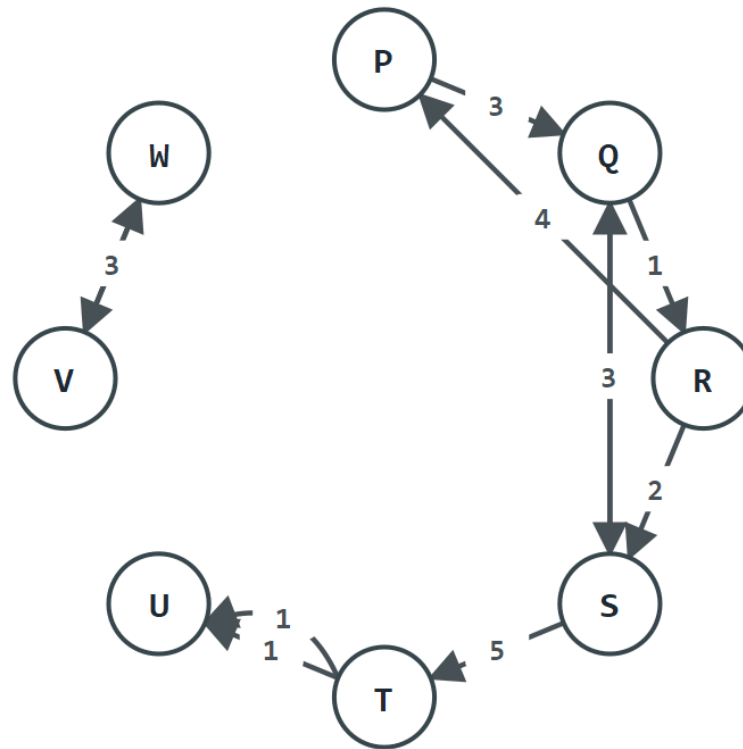
Operation	What it answers
<code>has_edge(a, b)</code>	Are <code>a</code> and <code>b</code> directly connected?
<code>neighbors(v)</code>	Which vertices does <code>v</code> connect to?
<code>degree(v)</code>	How many edges touch <code>v</code> ?

Graph vocabulary, one term at a time



Summary: 8 vertices, 10 edges, 2 component(s), a cycle exists.

Edge case — directed, multiple cycles, a self-loop



Summary: 8 vertices, 12 edges, 2 component(s), a cycle exists.

Code — the Edge and Graph structs

```
typedef struct Edge {  
    int to;           /* other endpoint */  
    int weight;       /* 1 if unweighted */  
    struct Edge *next;  
} Edge;  
  
typedef struct Graph {  
    Edge *adj[MAX_V]; /* one list per vertex */  
    int vertex_count;  
    int directed;     /* 0 or 1 */  
} Graph;
```

Code — out_degree()

```
int out_degree(Graph *g, int v) {  
    int d = 0;  
    for (Edge *e = g->adj[v]; e; e = e->next)  
        d++;  
    return d;  
}
```

Code — in_degree() (directed only)

```
int in_degree(Graph *g, int v) {  
    int d = 0;  
    for (int u = 0; u < g->vertex_count; u++)  
        for (Edge *e = g->adj[u]; e; e = e->next)  
            if (e->to == v) d++;  
    return d;  
}
```

Complexity

- `out_degree` : walk one list — $O(\text{degree of } v)$
- `in_degree` : scan **every** vertex's list — $O(V + E)$
- Directed graphs make "in" strictly more expensive than "out"

Common mistakes

- Confusing **degree** with **number of neighbors** on a multigraph
- Forgetting a self-loop counts **twice** toward degree
- Assuming in-degree and out-degree are always equal — only in special cases

Mini-quiz

A directed graph has a self-loop at vertex **x**.
By how much does that self-loop raise **x**'s
in-degree? Its out-degree?

Answer

By 1 each. A self-loop $x \rightarrow x$ counts once as an edge **into** x and once as an edge **out of** x — both in-degree and out-degree rise by 1.

3. Representing a Graph: Matrix vs. List

A question to start

`has_edge(a, b)` and `neighbors(v)` are the two questions every algorithm this week will ask.
What is the simplest structure that answers both?

Two representations at a glance

	Adjacency matrix	Adjacency list
Stores	A $V \times V$ table	One list per vertex
Best for	Dense graphs	Sparse graphs
<code>has_edge</code>	$O(1)$	$O(\text{degree})$

Adjacency matrix — the idea

A $V \times V$ table. `matrix[i][j]` holds **1** (or the weight) if an edge runs from vertex **i** to vertex **j**, **0** otherwise. Undirected graphs fill both `matrix[i][j]` and `matrix[j][i]`.

Adjacency matrix, step by step

	A	B	C	D	E	F	G
A	0	1	0	1	0	0	1
B	1	0	1	0	1	0	0
C	0	1	0	1	0	1	0
D	1	0	1	0	1	0	0
E	0	1	0	1	0	1	0
F	0	0	1	0	1	0	1
G	1	0	0	0	0	1	0

Done: the matrix is complete. Asking whether an edge exists is an $O(1)$ lookup, `matrix[i][j]`, but the table always takes V^2 cells as V grows, even when most of it is empty.

Edge case — a complete graph

	A	B	C	D	E
A	0	1	1	1	1
B	1	0	1	1	1
C	1	1	0	1	1
D	1	1	1	0	1
E	1	1	1	1	0

Done: the matrix is complete. Asking whether an edge exists is an $O(1)$ lookup, `matrix[i][j]`, but the table always takes V^2 cells as V grows, even when most of it is empty.

Code — add_edge() (matrix)

```
void add_edge(int a, int b, int w, int dir) {  
    matrix[a][b] = w;  
    if (!dir)  
        matrix[b][a] = w; /* mirror the diagonal */  
}
```

Code — build_adjacency_matrix()

```
void build_adjacency_matrix(Edge *edges, int n,  
                           int directed) {  
    for (int i = 0; i < MAX_V; i++)  
        for (int j = 0; j < MAX_V; j++)  
            matrix[i][j] = 0;  
    for (int k = 0; k < n; k++)  
        add_edge(edges[k].a, edges[k].b,  
                 edges[k].weight, directed);  
}
```

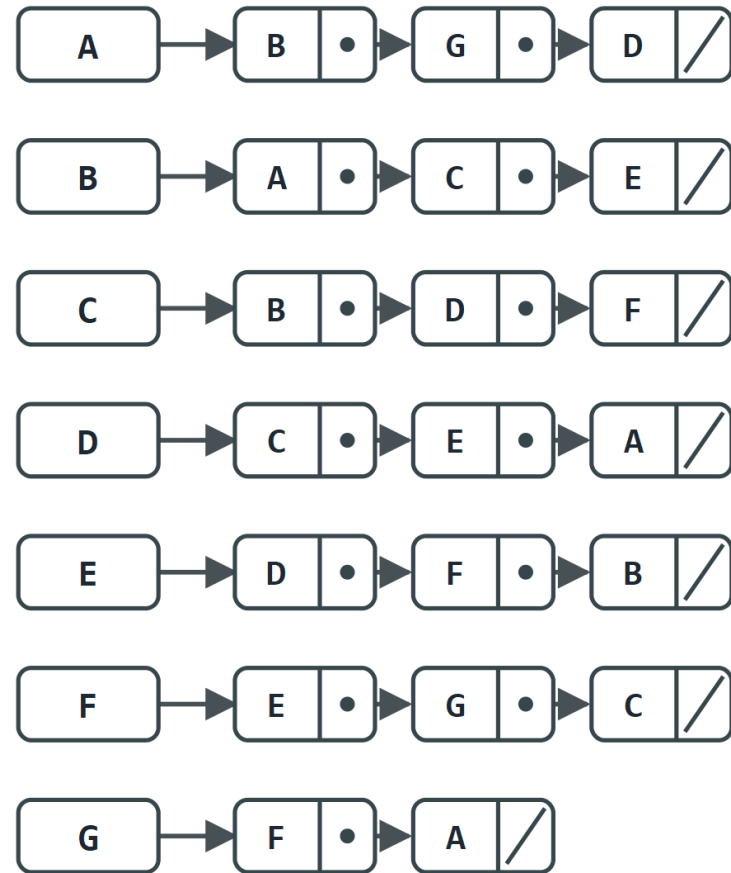
Complexity — matrix

- `has_edge(a, b)` : one lookup — $O(1)$
- Space: always $O(V^2)$ cells, dense or sparse alike
- `neighbors(v)` : scan a whole row — $O(V)$, even if v has 1 edge

Adjacency list — the idea

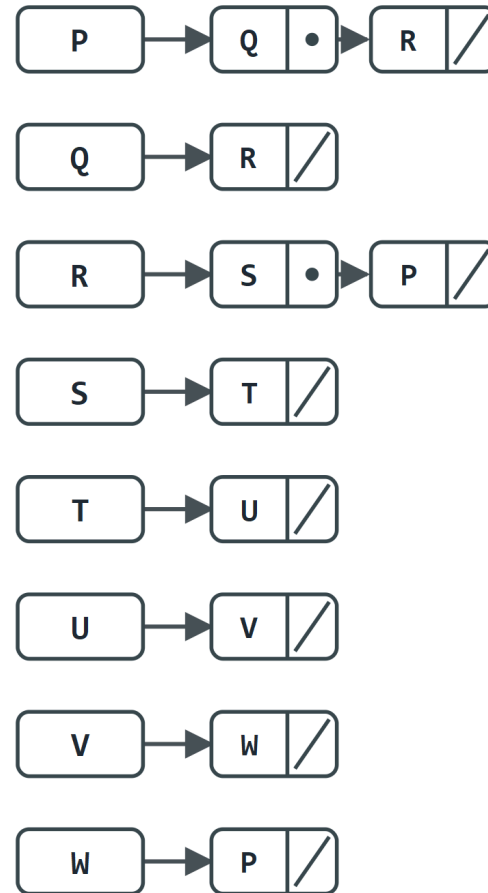
One linked list **per vertex**, holding only its actual neighbors. An undirected edge **A-B** appends **B** to **A**'s list **and** **A** to **B**'s list — two nodes, two lists.

Adjacency list, step by step



Done: 20 list nodes in total. An adjacency list only uses space for the edges that actually exist ($O(V + E)$); for a sparse graph it is far more economical than the matrix.

Edge case — directed, an empty list



Done: 10 list nodes in total. An adjacency list only uses space for the edges that actually exist ($O(V + E)$); for a sparse graph it is far more economical than the matrix.

Code — append() (list)

```
void append(int v, int neighbour) {  
    AdjNode *n = malloc(sizeof(AdjNode));  
    n->to = neighbour;  
    n->next = NULL;  
    if (adj[v] == NULL) { adj[v] = n; return; }  
    AdjNode *cur = adj[v];  
    while (cur->next) cur = cur->next;  
    cur->next = n;  
}
```

Code — add_edge() (list)

```
void add_edge(int a, int b, int directed) {  
    append(a, b);  
    if (!directed && a != b)  
        append(b, a);  
}
```

Complexity — list

- Space: $O(V + E)$ — only real edges take room
- `neighbors(v)` : walk its own list — $O(\text{degree of } v)$
- `has_edge(a, b)` : walk `a`'s list looking for `b` — $O(\text{degree})$

Memory & time, side by side

	Matrix	List
Space	$O(V^2)$	$O(V + E)$
has_edge	$O(1)$	$O(\text{degree})$
neighbors	$O(V)$	$O(\text{degree})$

Sparse graphs: pick the list

- A road map, the web, a friendship graph: all **sparse**
- E is close to V , far below V^2
- The list's $O(V + E)$ space stays small; the matrix wastes $O(V^2)$
- Every algorithm from Section 4 onward uses the **list**

Dense graphs: matrix can win

- A small, densely connected graph: E close to V^2
- The matrix's "wasted" $O(V^2)$ space is barely wasted at all
- `has_edge` at $O(1)$ can matter more than saving memory
- Rule of thumb: dense and small $\rightarrow$ matrix; sparse or large $\rightarrow$ list

Common mistakes

- Using a matrix for a large, sparse graph — wastes huge memory
- Forgetting the list needs **two** appends for one undirected edge
- Reading `matrix[a][b]` as symmetric on a **directed** graph

Mini-quiz

A graph has 1,000 vertices and only 2,000 edges. Roughly how many cells would its adjacency matrix need? Is that graph sparse?

Answer

About 1,000,000 cells (1000^2), for only 2,000 real edges — almost all of it wasted.

Yes, **very sparse**: an adjacency list is the clear choice here.

4. Breadth-First Search (BFS)

A question to start

Starting from one person, who are their direct friends? Their friends-of-friends? Visit everyone, **nearest first** — how?

A short history

- **1959** — Edward F. Moore publishes "The shortest path through a maze"
- Independently rediscovered soon after, in different fields
- The queue-based method here is essentially Moore's algorithm
- Now one of the most widely used graph algorithms in practice

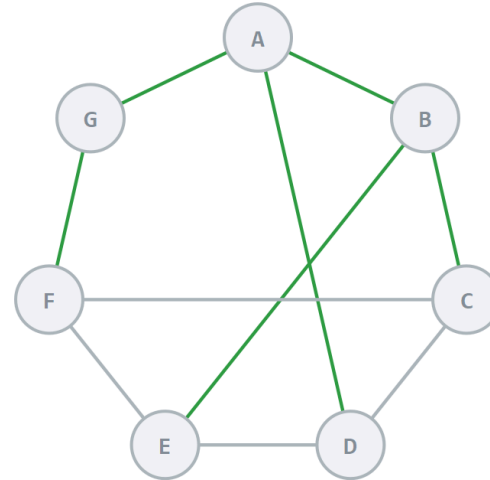
Intuition — ripples on a pond

- Drop a stone: the first ripple reaches the nearest points
- The next ripple reaches everything one step further out
- BFS visits a graph the same way — ring by ring, outward
- Each "ring" here is called a **level**

The BFS idea

- Use a **queue** (FIFO) — Week 3's structure, holding vertices
- Start vertex: enqueue it, mark visited, level 0
- Dequeue a vertex, enqueue its unvisited neighbors, level + 1
- The visited edges form a **BFS tree**, rooted at the start

BFS, step by step



level[] =

A	B	C	D	E	F	G
0	1	2	1	2	2	1

queue =

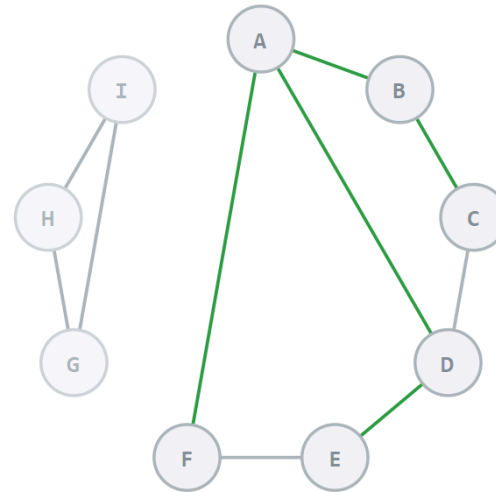
--	--	--	--	--	--	--

visit order =

A	B	D	G	C	E	F
---	---	---	---	---	---	---

The queue is empty, done. Full order: A, B, D, G, C, E, F -- every vertex was reached by a path with the fewest possible edges.

Edge case — a disconnected graph



level[] =

A	B	C	D	E	F	G	H	I
0	1	2	1	2	1			

queue =

--	--	--	--	--	--	--	--	--

visit order =

A	B	D	F	C	E
---	---	---	---	---	---

The queue is empty. G, H, I cannot be reached from A by any edge -- they stay grey. Full order: A, B, D, F, C, E.

Code — enqueue / dequeue

```
void enqueue(int v) {
    queue_data[rear] = v;
    rear = (rear + 1) % MAX_V;
    count++;
}
int dequeue(void) {
    int v = queue_data[front];
    front = (front + 1) % MAX_V;
    count--;
    return v;
}
```

Code — bfs() main loop

```
visited[start] = 1;
level_of[start] = 0;
enqueue(start);
while (count > 0) {
    int u = dequeue();
    for (Edge *e = g->adj[u]; e; e = e->next) {
        if (!visited[e->to]) {
            visited[e->to] = 1;
            level_of[e->to] = level_of[u] + 1;
            parent_of[e->to] = u;
            enqueue(e->to);
        }
    }
}
```

Complexity

- Every vertex enqueued once, dequeued once — $O(V)$
- Every edge examined at most twice (once per endpoint) — $O(E)$
- Total: $O(V + E)$ — the same bound as every list operation

BFS tree and shortest paths

- The BFS tree's root-to-vertex path uses the **fewest edges**
- `level_of[v]` is exactly that shortest edge-count from the start
- This only holds for **unweighted** graphs — Section 8 uses it directly
- Weighted shortest paths need Dijkstra's algorithm, Week 9

Applications

- "Friends of friends" — Section 4's social-network motivation
- Web crawling level by level, nearest pages first
- Puzzle solvers: fewest moves to solve a maze or a sliding puzzle
- Network broadcast: reach every machine in the fewest hops

Common mistakes

- Using a **stack** instead of a queue — silently becomes depth-first
- Forgetting to mark a vertex visited **at enqueue time**, not dequeue
- Re-enqueuing an already-visited vertex — wastes work, may loop

Mini-quiz

In an unweighted graph, BFS from **A** gives **B** level 2. How many edges are on the **shortest** path from **A** to **B**?

Answer

Exactly 2 edges. BFS's level is defined as the fewest edges from the start — that is precisely the shortest-path edge count.

5. Depth-First Search (DFS) — Recursive

A question to start

A maze has many branching paths. Instead of exploring every direction a little at a time, what if you committed to **one** path, all the way, before ever backtracking?

Intuition — a maze, go deep first

- Pick a direction, walk it as far as it goes
- Dead end? Back up to the last choice, try the next direction
- "Back up" is exactly what a **stack** (or recursion) does for you
- Depth-first, not breadth-first — one branch, fully, before the next

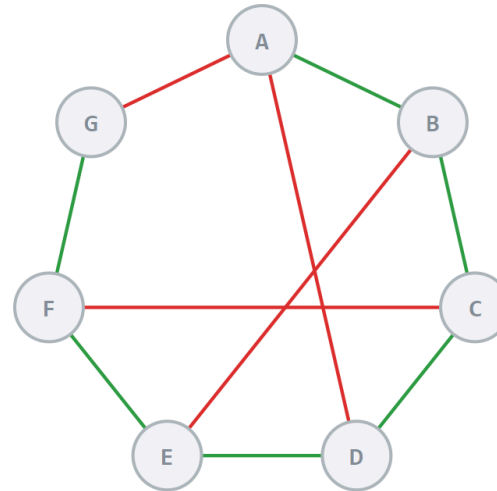
Colors and times

- **White** (0): not yet discovered
- **Gray** (1): discovered, still exploring its neighbors
- **Black** (2): finished, every neighbor explored
- `disc[v]` / `fin[v]`: the clock tick each color change happens at

Edge types

Type	Meaning
Tree edge	Leads to a new, white vertex
Back edge	Leads to a gray ancestor — a cycle!
Forward / cross	Leads to a finished, black vertex (directed only)

DFS recursive, step by step



disc/fin[] =

A	B	C	D	E	F	G
1/14	2/13	3/12	4/11	5/10	6/9	7/8

call stack =

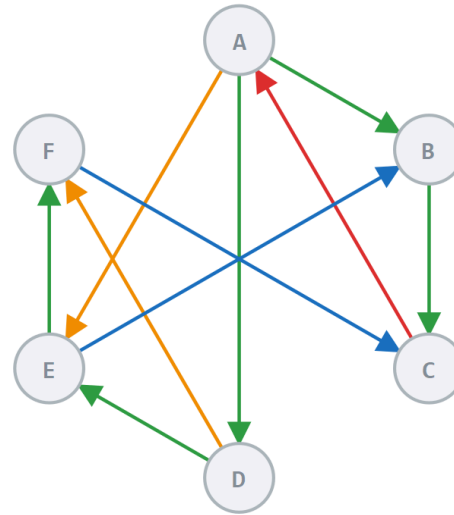
--	--	--	--	--	--	--

visit order =

A	B	C	D	E	F	G
---	---	---	---	---	---	---

Done. Visit order: A, B, C, D, E, F, G.

Edge case — all four edge types at once



disc/fin[] =

A	B	C	D	E	F
1/12	2/5	3/4	6/11	7/10	8/9

call stack =

--	--	--	--	--	--

visit order =

A	B	C	D	E	F
---	---	---	---	---	---

Done. Visit order: A, B, C, D, E, F.

Code — dfs_visit(): discovery

```
void dfs_visit(Graph *g, int u) {
    color_of[u] = 1;          /* gray */
    disc_time[u] = ++clock_;
    for (Edge *e = g->adj[u]; e; e = e->next) {
        int v = e->to;
        /* ... classify the edge to v ... */
    }
    color_of[u] = 2;          /* black */
    fin_time[u] = ++clock_;
}
```

Code — classifying an edge

```
if (color_of[v] == 0) {  
    parent_of[v] = u;  
    dfs_visit(g, v);    /* tree edge */  
} else if (color_of[v] == 1) {  
    /* back edge: v is an ancestor */  
} else {  
    /* v is black: forward or cross */  
}
```

Code — one tree per component

```
void dfs(Graph *g) {  
    for (int i = 0; i < g->vertex_count; i++)  
        color_of[i] = 0;  
    for (int i = 0; i < g->vertex_count; i++)  
        if (color_of[i] == 0)  
            dfs_visit(g, i);  
}
```

Complexity

- Every vertex colored white $\rightarrow$ gray $\rightarrow$ black **once** — $O(V)$
- Every edge examined exactly once (twice if undirected) — $O(E)$
- Total: **$O(V + E)$** — identical to BFS's bound

Recursion depth: a hidden risk

- Each `dfs_visit` call adds one frame to the **call stack**
- A long chain graph can recurse as deep as `v` calls
- A very large or very "stringy" graph can **overflow** that stack
- Section 6 rebuilds DFS with an explicit stack to avoid exactly this

Common mistakes

- Forgetting to skip the edge straight back to your own **parent**
- Treating every non-tree edge as a "cycle" on a **directed** graph
- Reusing `color_of` across runs without resetting it to white

Mini-quiz

In an undirected graph, DFS finds an edge to a vertex that is currently **gray**. What does that tell you about the graph?

Answer

The graph has a **cycle**. A gray vertex is still on the call stack — it is an **ancestor** of the current vertex, so this edge is a back edge.

6. DFS — Iterative (Explicit Stack)

A question to start

Section 5 ended on a warning: deep recursion can overflow the call stack. Can the *exact same* traversal be written without recursion?

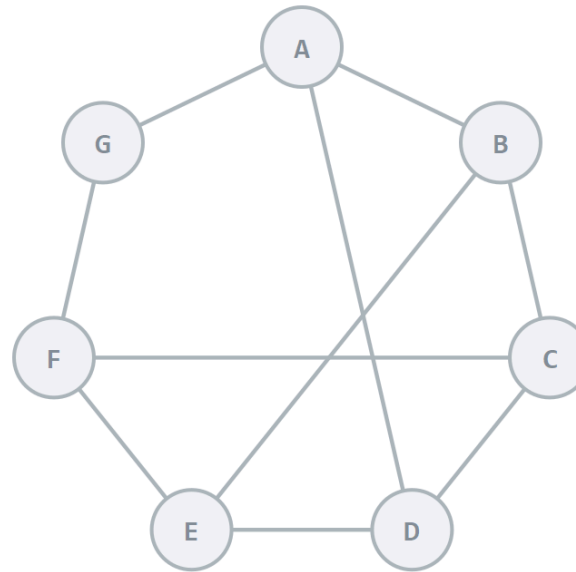
Why go iterative?

- Recursion's call stack has a fixed size, set by the OS
- Our **own** stack (Week 3's array-based one) can be sized as needed
- Same algorithm, same visit order — only the bookkeeping moves
- No memory saved overall, just moved somewhere safer

The reverse-order trick

- Neighbors must be visited in **alphabetical** order, like recursion
- Pushing them alphabetically would pop them **backwards**
- Fix: push neighbors in **reverse** alphabetical order
- Popping a reversed-push order restores the original order

DFS iterative, step by step

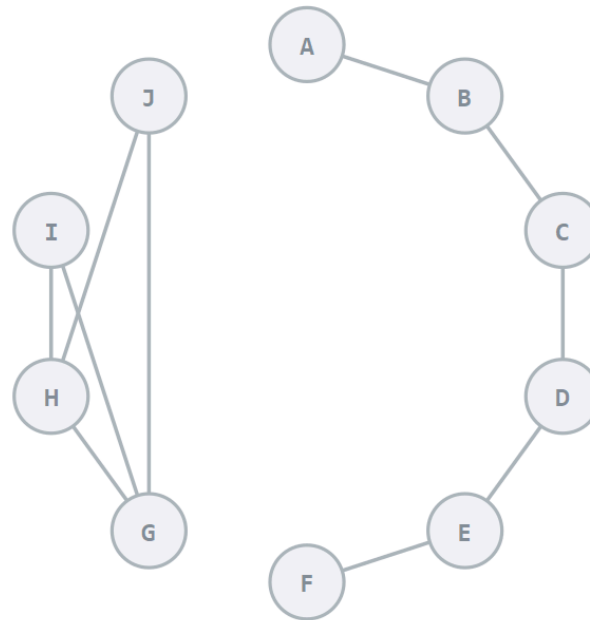


stack =

visit order = A B C D E F G

The stack is empty, done. Visit order: A, B, C, D, E, F, G -- exactly the same as the recursive version.

Edge case — a DFS forest



stack =

visit order = A B C D E F G H I J

The stack is empty, done. Visit order: A, B, C, D, E, F, G, H, I, J -- exactly the same as the recursive version.

Code — push / pop

```
void push(int v) { stack_data[++top] = v; }  
int pop(void)   { return stack_data[top--]; }
```

Code — dfs_iterative() main loop

```
push(start);
while (top >= 0) {
    int u = pop();
    if (visited[u]) continue; /* stale entry */
    visited[u] = 1;
    for (int i = deg[u] - 1; i >= 0; i--)
        if (!visited[adj[u][i]])
            push(adj[u][i]);
}
```

Complexity

- Every vertex pushed and popped a bounded number of times — $O(V)$
- Every edge examined once (twice if undirected) — $O(E)$
- Total: $O(V + E)$ — no better, no worse than the recursive version

Recursive vs. iterative, side by side

	Recursive (Sec. 5)	Iterative (Sec. 6)
Stack	The call stack	Our own array
Visit order	Alphabetical	Alphabetical (reversed pushes)
Deep-graph risk	Can overflow	Bounded by array size

Common mistakes

- Pushing neighbors in **forward**, not reverse, alphabetical order
- Forgetting the stale-entry check — double-visits, wrong disc/fin
- Assuming a popped, unvisited vertex is always freshly discovered

Mini-quiz

A vertex is pushed **twice**, from two different neighbors, before either pop happens. How many times is it actually visited?

Answer

Once. The first `pop()` visits it and marks it visited; the second `pop()` finds `visited[u]` already true and is discarded.

7. Connected Components

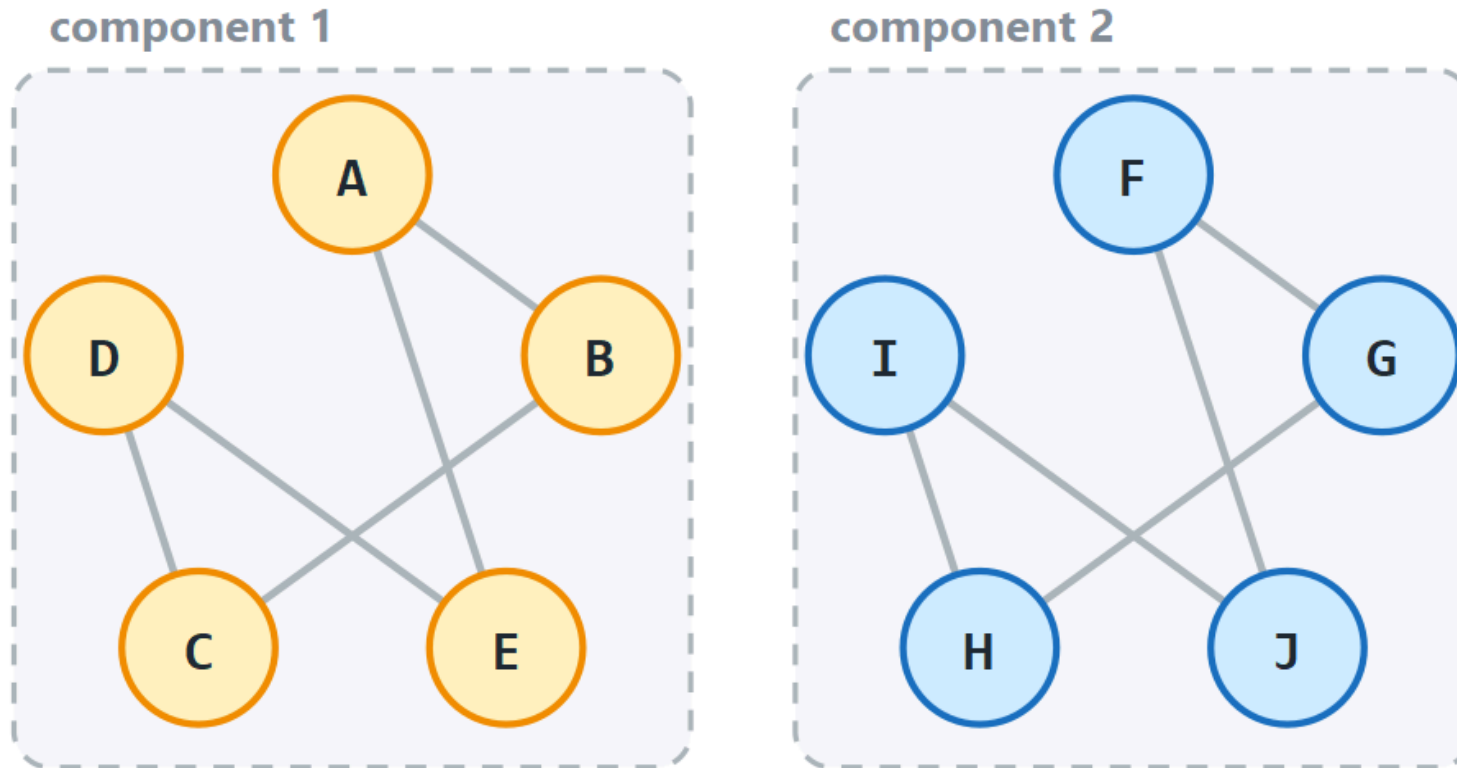
A question to start

A graph may not be one connected piece —
it can be several separate pieces. How many?
Which vertices belong to which piece?

The idea

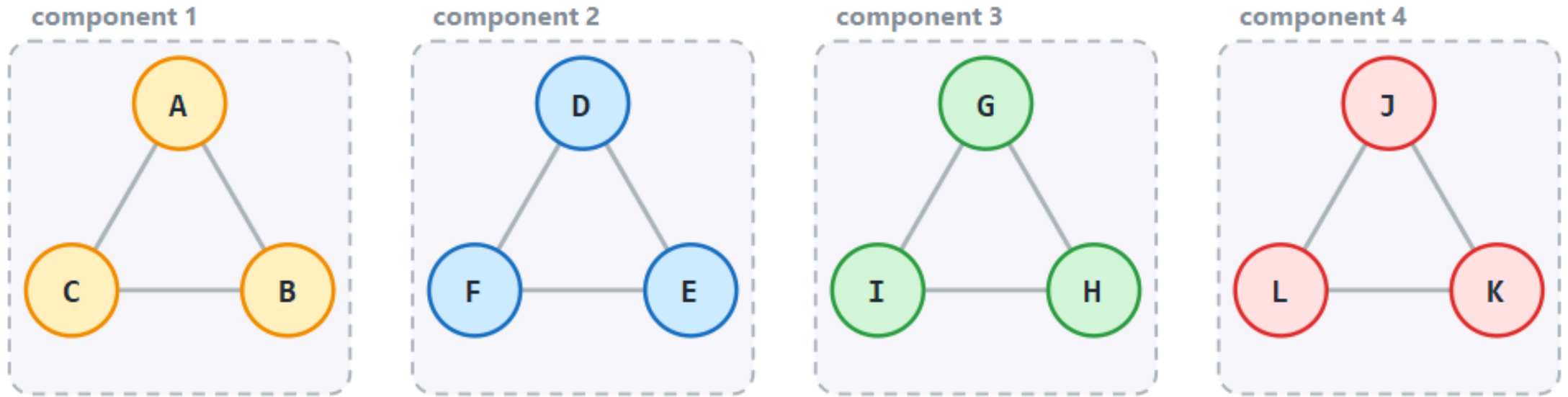
- Every unvisited vertex starts a **new** BFS, with a new id
- That BFS labels everything **it can reach** with that same id
- Once the queue empties, the next unvisited vertex starts the next
- Direction is **ignored** — this finds **weak** connectivity

Connected components, step by step



Done: 2 connected component(s).

Edge case — many small components



Done: 4 connected component(s).

Code — bfs_label()

```
void bfs_label(Graph *g, int start, int id) {  
    int q[MAX_V], front = 0, rear = 0;  
    comp_of[start] = id;  
    q[rear++] = start;  
    while (front < rear) {  
        int u = q[front++];  
        for (Edge *e = g->adj[u]; e; e = e->next)  
            if (comp_of[e->to] == -1) {  
                comp_of[e->to] = id;  
                q[rear++] = e->to;  
            }  
    }  
}
```

Code — count_components()

```
int count_components(Graph *g) {  
    int next_id = 0;  
    for (int i = 0; i < g->vertex_count; i++)  
        comp_of[i] = -1;  
    for (int i = 0; i < g->vertex_count; i++)  
        if (comp_of[i] == -1)  
            bfs_label(g, i, next_id++);  
    return next_id;  
}
```

Complexity

- Every vertex enters exactly one `bfs_label` call — $O(V)$
- Every edge examined at most twice total, across all calls — $O(E)$
- Total: $O(V + E)$ — the cost of one BFS, done once per component

Applications

- Social networks: which friend circles exist, and who is in each
- Image processing: which pixels form one connected region
- Networking: which machines can currently reach each other
- Puzzle games: "flood fill" — the paint-bucket tool, essentially

Common mistakes

- Forgetting direction is **ignored** here — this is weak connectivity
- Reusing `comp_of` values across separate graphs without resetting
- Assuming a single-vertex graph has **zero** components — it has one

Mini-quiz

A directed graph has an edge $A \rightarrow B$ but no edge $B \rightarrow A$. Weakly, are A and B in the same component?

Answer

Yes. Weak connectivity ignores direction entirely, so $A > B$ alone is enough to place A and B in the same component.

8. Shortest Path in Unweighted Graphs

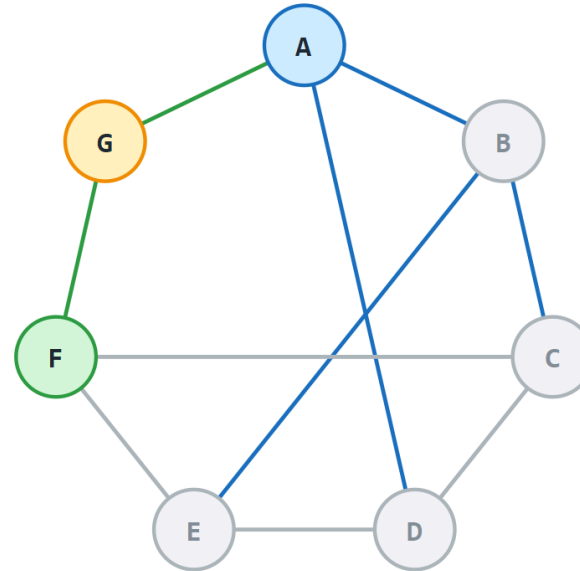
A question to start

BFS already knows the fewest edges from **A** to every vertex. Can it also report the **exact route** — not just the distance?

The idea

- Run BFS from `s`, recording each vertex's **parent** as usual
- To find the path to `t`: start at `t`, follow **parent** backward
- Stop when you reach `s` — then **reverse** the collected list
- The result is the shortest `s`-to-`t` path, by edge count

Path-finding BFS, step by step



parent[] =

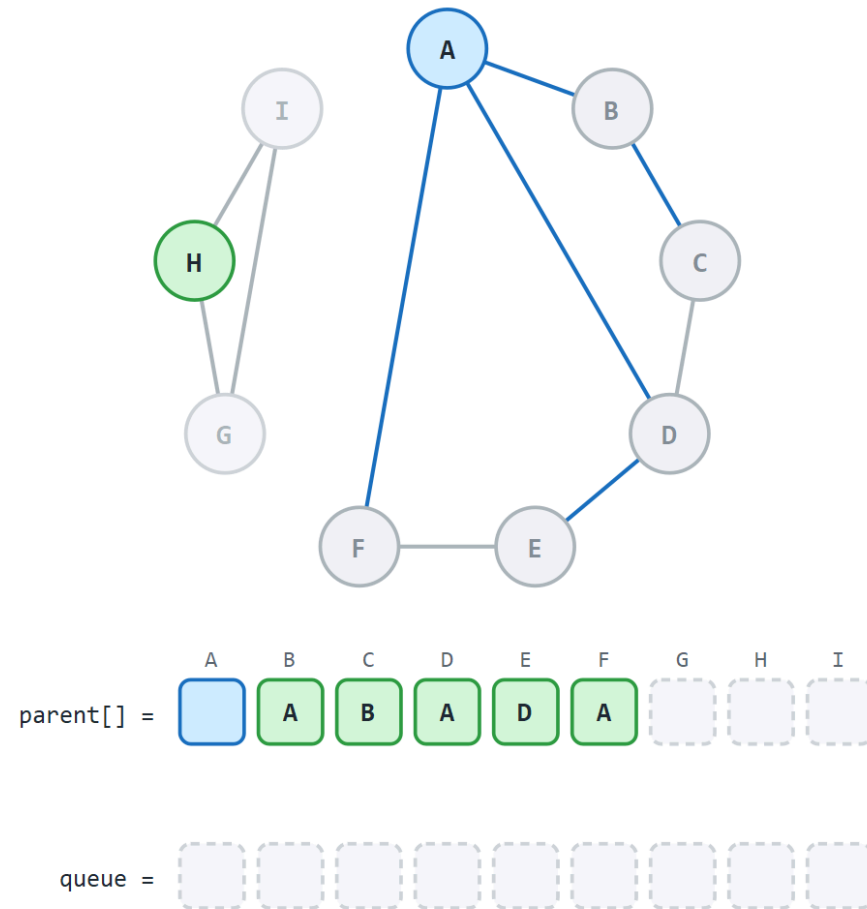
A	B	C	D	E	F	G
A	A	B	A	B	G	A

queue =

--	--	--	--	--	--	--

Walking the parent pointers back from F to A: A → G → F. Length (edge count): 2.

Edge case — no path exists



The queue is empty but H was never visited: there is NO PATH from A to H.

Code — bfs_shortest_path(): the search

```
visited[s] = 1;
queue_data[rear++] = s;
while (front < rear) {
    int u = queue_data[front++];
    for (Edge *e = g->adj[u]; e; e = e->next)
        if (!visited[e->to]) {
            visited[e->to] = 1;
            parent_of[e->to] = u;
            queue_data[rear++] = e->to;
        }
}
```

Code — walking back the path

```
if (!visited[t]) return -1;      /* no path */
int len = 0, v = t;
while (v != s) {
    path_out[len++] = v;
    v = parent_of[v];
}
path_out[len++] = s;
/* ... reverse path_out[0..len) in place ... */
return len - 1;                 /* edges */
```

Complexity

- The BFS itself: $O(V + E)$, exactly as in Section 4
- Walking the path back: $O(\text{path length})$, at most $O(V)$
- Total: still $O(V + E)$ — the walk-back never dominates

Multiple shortest paths can exist

- BFS reports **one** shortest path, not necessarily the only one
- A different neighbor-visiting order can produce a different, equally short path
- All correct shortest paths share the same **length**, never necessarily the same route

Preview: weighted graphs need Dijkstra

- This section only works because every edge secretly has weight 1
- A **weighted** graph needs to compare actual distances, not edge counts
- **Dijkstra's algorithm** (Week 9) solves that, using a **priority queue**
- That priority queue is exactly the one Week 4 built from a heap

Common mistakes

- Forgetting to check `visited[t]` before walking the path back
- Walking `parent` forward instead of backward, from `t` to `s`
- Forgetting the final reversal — the path prints from `t` to `s`

Mini-quiz

BFS from `s` never visits `t` at all. What should `bfs_shortest_path` return, and why?

Answer

-1. If `t` was never visited, no path exists at all — the `visited[t]` check catches exactly this case before any walk-back begins.

Summary — vocabulary and representations

Idea	Key fact
Graph	V vertices, E edges, directed or not
Degree	Undirected: 1 value; directed: in and out
Matrix	$O(V^2)$ space, $O(1)$ <code>has_edge</code>
List	$O(V + E)$ space, $O(\text{degree})$ <code>has_edge</code>

Summary — traversals and applications

Idea	Key fact
BFS	Queue, level order, shortest unweighted path
DFS (recursive / iterative)	Stack, deep first, same visit order both ways
Components	Repeated BFS, one label per reachable set
Shortest path	BFS + parent pointers, walked backward

The big picture

Drop a tree's "one root, no cycles" rule and you get a graph. Two structures store it; two traversals, BFS and DFS, explore it; both answer real questions — components, distance.

Self-check round

Four short questions. Think before the answer appears on the next slide. Full exercises and a ten-question quiz are in the week notes.

1. A graph has 6 vertices and is a complete graph (every pair connected). How many edges does it have?

$6 * 5 / 2 = 15$ edges — every pair counted once, since the graph is undirected.

2. Why does BFS need a queue instead of a stack?

A queue's FIFO order visits the nearest vertices first; a stack would go deep immediately, becoming DFS instead.

3. What does a "back edge" during DFS tell you about the graph?

The graph contains a cycle — a back edge always points to a gray ancestor still on the call stack.

4. Connected components treats a directed graph as if it were undirected. What is this called?

Weak connectivity — direction is ignored; strong connectivity, requiring a path back too, is a harder, different question.

Next week

Week 6 — Search and Hashing

BFS and DFS both walk an entire graph to find something. What if you could look an item up directly, in close to $O(1)$ time?

References (1/2)

- Course syllabus, Week 5: `docs/syllabus/syllabus.en.md`
- Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 4th ed. MIT Press
- Sedgewick, Wayne. *Algorithms*, 4th ed. Addison-Wesley
- Euler, L. (1736). *Solutio problematis ad geometriam situs pertinentis*

References (2/2)

- Moore, E. F. (1959). "The shortest path through a maze"
- Hopcroft, J., Tarjan, R. (1973) — DFS edge classification
- Knuth. *The Art of Computer Programming, Vol. 1*, 3rd ed.
- [williamfiset/Algorithms](#) · Programiz DSA